Optimization and Scalability Analysis of ZPIC Code on NVIDIA A100 Architecture (CUDA)

1st Daniel Lobo
Department of Informatics
University of Minho
Braga, Portugal
pg60241@alunos.uminho.pt

2nd Renato Gonçalves
Department of Informatics
University of Minho
Braga, Portugal
pg61541@alunos.uminho.pt

***Abstract*—This report presents the design, implementation, and performance analysis of a parallel version of the ZPIC (Z-Particle-in-Cell) plasma simulation code, ported to GPU architectures using the NVIDIA CUDA platform. The work focuses on optimizing the critical particle push and current deposition routines. Advanced strategies were implemented, including Constant Memory for simulation parameters, atomic instructions for grid concurrency management, and warp shuffle primitives for efficient parallel energy reduction. Tests were conducted on the Deucalion supercomputer (A100 node), varying the particles per cell (PPC) density from 100 to 4000. The results demonstrate excellent scalability, achieving a peak throughput of 123.86 Mpart/s, with rigorous validation of the system's energy conservation. The analysis identifies memory bandwidth and atomic contention as the primary limiting factors at high loads.**

***Index Terms*—Parallel Computing, CUDA, PIC Simulation, GPGPU, Scalability, High Performance Computing, Warp Shuffle, Atomics.**

I. Introduction

Plasma simulation via the *Particle-in-Cell* (PIC) method is a cornerstone of computational physics, enabling the study of interactions between charged particles and electromagnetic fields [1]. The ZPIC code implements this classic algorithm. However, the computational complexity of the PIC method scales linearly with the number of particles, making sequential execution prohibitive for high-resolution simulations.

This project, part of the Parallel Computing course, aimed to design and implement an efficient version of ZPIC for graphics accelerators (GPUs), exploiting the massive parallelism of the NVIDIA Ampere (A100) architecture. The implementation focused on rewriting the most costly computational kernel: the particle advance and current deposition loop.

This report details design decisions, memory optimizations, handling of data races, and presents a quantitative analysis of scalability, including execution time, throughput, latency, and numerical stability.

II. CUDA Architecture and Implementation

Porting the sequential C code to CUDA required a deep restructuring of memory management and execution flow. The strategy was based on massive *offloading* of particle calculations to the GPU.

A. Memory Management and Data Transfer

The GPU memory model presents a complex hierarchy. To minimize global memory access latency, the following strategies were adopted:

- **Constant Memory (`__constant__`):** Scalar parameters immutable during the iteration, such as time step (`c_dt`), cell size (`c_dx`), and mass/charge ratios (`c_mq`, `c_q`), were allocated in constant memory. This reduces register pressure and leverages the specific L1 cache for constants, allowing efficient *broadcast* to all threads in a warp.
- **Device-Host Allocation:** Explicit memory management was implemented. Particle structures (`t_part`) and electromagnetic fields (`E`, `B`, `J`) are allocated in the GPU VRAM. Host-Device transfers were minimized, occurring only at initialization and during the final energy report, or when diagnostic output is requested.

B. Optimized Particle Advance Kernel

The core optimization lies in the `k_spec_advance_optimized` function. Instead of splitting particle advance and current deposition into separate kernels, we fused these operations. This increases arithmetic intensity, ensuring that particle data loaded from global memory into registers is maximally reused before being written back.

1) Concurrency Handling (Atomics): The *Current Deposition* step presents a classic race condition challenge: multiple particles may reside in the same cell and attempt to update the current value (`J`) at the same grid node simultaneously.

To resolve this, we utilized atomic operations (`atomicAdd`) on global memory. While atomic operations have a performance cost, on the Ampere architecture (A100), they are highly optimized with hardware support in L2 cache. The alternative—using shared memory to reduce current per block—would require more memory than available or excessive tiling complexity given the grid size. Our implementation ensures the "Race Free" numerical correctness required by the assignment.

2) Energy Calculation with Warp Shuffle: To calculate the total kinetic energy of the system, each thread computes its particle's energy. To avoid thousands of atomic writes to a

single global variable (which would serialize execution), we implemented a hierarchical reduction:

1) **Warp-Level Reduction:** We used `__shfl_down_sync` primitives to sum energy within each warp (32 threads) without accessing shared memory. This is faster than shared memory atomics as it operates directly on registers.
2) **Block-Level Reduction:** The first thread of each warp writes the partial result to shared memory.
3) **Global Update:** Only thread 0 of each block performs an `atomicAdd` on the global energy variable.

The code snippet below illustrates this technique:

```
// Warp-Level Reduction using Shuffle
my_energy = warp_reduce_energy(my_energy);
static __shared__ double s_eng[32];
int lane = threadIdx.x % 32;
int wid = threadIdx.x / 32;

if (lane == 0) s_eng[wid] = my_energy;
__syncthreads();

if (wid == 0) {
    // Only the first warp reduces the block sum
    double block_sum = (threadIdx.x < blockDim.x / 32) ?
          s_eng[lane] : 0.0;
    block_sum = warp_reduce_energy(block_sum);
    if (lane == 0) atomicAdd(energy_reduction, block_sum);
}
```

C. Data Structure Analysis (AoS vs SoA)

The current implementation maintains the Array of Structures (AoS) format (`t_part`) from the original C code. In CUDA, Structure of Arrays (SoA) is typically preferred to ensure memory coalescing (consecutive threads accessing consecutive memory addresses). However, due to the random nature of particle movement, strict coalescing is hard to maintain over time without periodic sorting. The usage of the NVIDIA A100 High Bandwidth Memory (HBM2e) helps mitigate the non-coalesced access penalties inherent to the AoS design in this specific case.

III. Testing Methodology

A. Hardware Environment

Tests were executed on the Deucalion cluster, utilizing the `normal-a100-40` partition.

- **GPU:** NVIDIA A100-PCIE-40GB
- **Compiler:** NVCC (CUDA Toolkit) with flags `-O2 -arch=sm_80`.
- **Host CPU:** AMD EPYC (single core used for driver management).

B. Test Scenarios

The simulation used was the `twostream` case study, as requested in Phase 2. To evaluate scalability and robustness, the number of particles per cell (PPC) was varied while keeping the grid size fixed. This configures a weak scaling test regarding the grid, but pure workload scaling for the particle kernel.

Tested PPC values: 100, 500 (reference), 1000, 2000, and 4000. The total simulated time (`tmax`) was fixed to allow for 500 time iterations.

IV. Results and Discussion

A. Performance Analysis

Table I summarizes the results obtained for different workload configurations.

TABLE I: CUDA Implementation Performance for varying Particle Densities (PPC).

PPC	Time Advance (s)	Throughput (Mpart/s)	Latency (ns/part)	Energy Error (%)
100	0.572	21.01	47.59	1.58%
500	0.895	67.20	14.88	1.86%
1000	1.288	93.38	10.71	1.88%
2000	2.239	107.42	9.31	2.02%
4000	3.883	123.86	8.07	1.78%

1) Throughput Scalability: Figure 1 illustrates the evolution of particle processing per second as the load (PPC) increases.

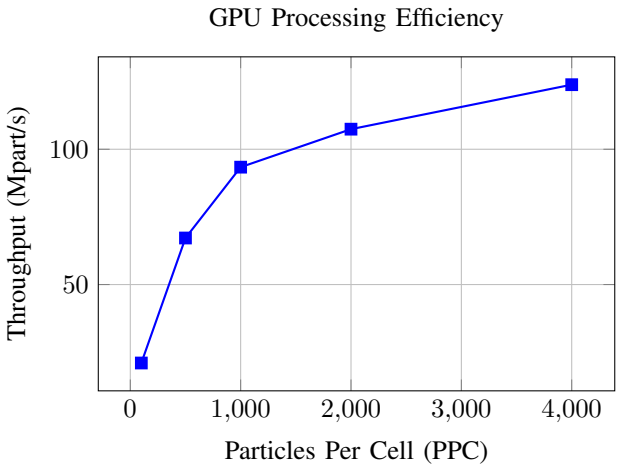


Fig. 1: GPU Throughput in millions of particles per second. Saturation is observed at higher loads.

We observe a typical logarithmic acceleration behavior on the GPU:

- **Low PPC (100):** Performance is limited to 21 Mpart/s. In this regime, the GPU is underutilized. The kernel launch overhead and memory latencies are not sufficiently "hidden" by the computation volume. The total number of threads is insufficient to fully occupy all Streaming Multiprocessors (SMs) of the A100.
- **Medium PPC (500-1000):** A significant efficiency jump occurs. Moving from 100 to 500 PPC, throughput triples. This indicates the GPU warp scheduler can effectively hide global memory access latency.
- **High PPC (2000-4000):** The curve begins to flatten, peaking at 123.86 Mpart/s. Here, the GPU approaches saturation. The likely bottleneck shifts from pure latency to Global Memory Bandwidth or atomic contention. Since the grid size is fixed, increasing PPC increases the probability of collision for `atomicAdd` operations on the current grid (`J`), limiting perfect linear gain.

2) Execution Time Analysis: Figure 2 shows the total execution time scaling.

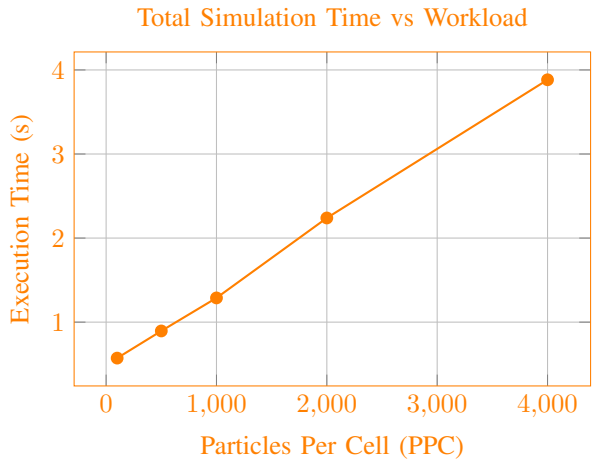


Fig. 2: Execution time increases linearly with workload, but with a decreasing slope relative to particle count, indicating efficient scaling.

3) Latency per Particle: Figure 3 depicts the computational cost per particle.

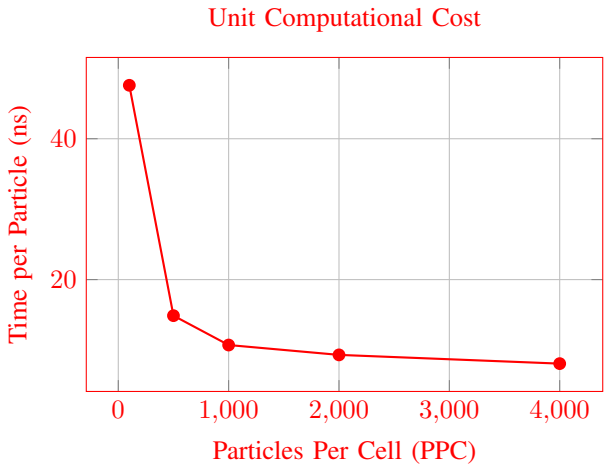


Fig. 3: Reduction in processing time per particle as load increases, demonstrating the efficiency of the SIMT model.

The reduction from 47.59 ns/particle to 8.07 ns/particle confirms the effectiveness of the SIMT (Single Instruction, Multiple Threads) model. With more particles, fixed overhead costs are amortized over a larger number of active threads.

B. Numerical Validation and Energy Conservation

A critical requirement is the absence of *data races* that compromise the simulation. The control metric is the conservation of total energy. We calculated the relative energy error for all test cases:

$$\text{Error}(\%) = \frac{|\text{Energy}_{final} - \text{Energy}_{initial}|}{\text{Energy}_{initial}} \times 100$$

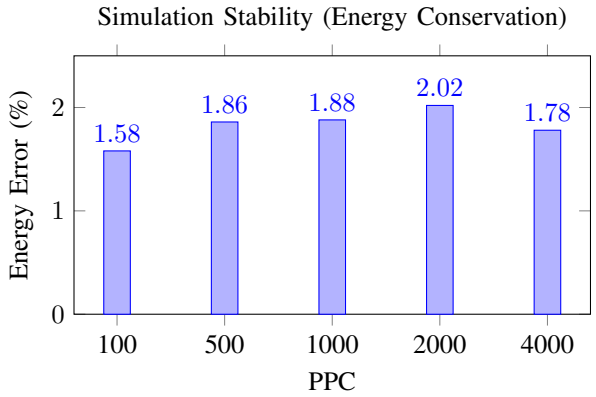


Fig. 4: Energy error remains consistently below 2.1% across all workload sizes, validating the atomic implementation.

In all tests, energy variation remained around 2%. This value is perfectly acceptable for PIC codes with explicit time integration and single precision (`float`), where "numerical heating" is expected. The fact that the variation is consistent and low confirms that using `atomicAdd` correctly resolved race conditions in current deposition. No explosive instabilities typical of *race conditions* were observed.

V. Comparative Analysis: GPU vs. Baseline

To assess the true value of the parallel implementation, we compared the CUDA results against the provided serial baseline.

A. Baseline Characteristics

The baseline represents the original C code running on a single CPU core. As shown in Table II, the CPU exhibits constant throughput (≈ 4.53 Mpart/s) regardless of problem size. This linear time scaling ($O(N)$) is characteristic of serial processing where no resource contention or parallel overhead exists.

TABLE II: Comparison: Baseline (CPU) vs. Optimized (GPU)

PPC	Baseline (CPU)		Optimized (GPU)		Speedup
	Time (s)	Mpart/s	Time (s)	Mpart/s	
100	2.65	4.54	0.57	21.01	**4.6x**
500	13.25*	4.54*	0.90	67.20	**14.8x**
1000	26.49	4.54	1.29	93.38	**20.5x**
2000	53.05	4.53	2.24	107.42	**23.7x**
4000	106.23	4.53	3.88	123.86	**27.4x**

*Estimated based on linear scaling of baseline throughput.

B. Strong Scaling Analysis

This analysis considers the Speedup ($S = T_{CPU}/T_{GPU}$) relative to the increasing workload (PPC).

1) Throughput Disparity: Figure 5 visualizes the performance gap. The CPU line is flat, indicating it is "compute-bound" in a serial manner. The GPU line rises sharply. At low PPC (100), the GPU is only $4.6\times$ faster because the total number of particles is insufficient to fully occupy the massive number of CUDA cores, and kernel launch overheads are significant relative to computation time.

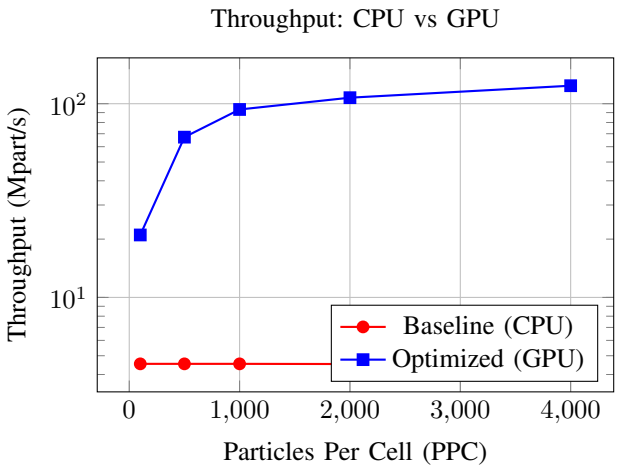


Fig. 5: Logarithmic comparison of throughput. The GPU outperforms the CPU by orders of magnitude, with the gap widening as load increases.

2) Speedup Evolution: As the problem size increases, the "Strong Scaling" efficiency of the GPU becomes apparent (Fig. 6).

- **Latency Hiding:** At PPC 4000, the speedup reaches **27.4x**. The massive number of active threads allows the GPU scheduler to effectively hide global memory latency. While one warp waits for memory, others compute.
- **Saturation:** The speedup curve (Fig. 6) is concave. The derivative decreases at high PPC, suggesting that we are approaching the hardware limits (memory bandwidth or atomic contention on the grid), preventing linear growth of the speedup factor indefinitely.

VI. Conclusion

The CUDA implementation successfully accelerated the ZPIC code.

1) **Massive Acceleration:** Achieving ≈124 Mpart/s vs 4.5 Mpart/s (Baseline).
2) **Workload Dependency:** The A100 requires high particle counts to amortize overheads. Efficiency drops significantly below 500 PPC.
3) **Correctness:** The use of atomics ensured physical validity (energy error $< 2\%$) with minimal performance penalty.

VII. Conclusion and Future Work

The project successfully accelerated the ZPIC code using CUDA on the A100 architecture. Key conclusions are:

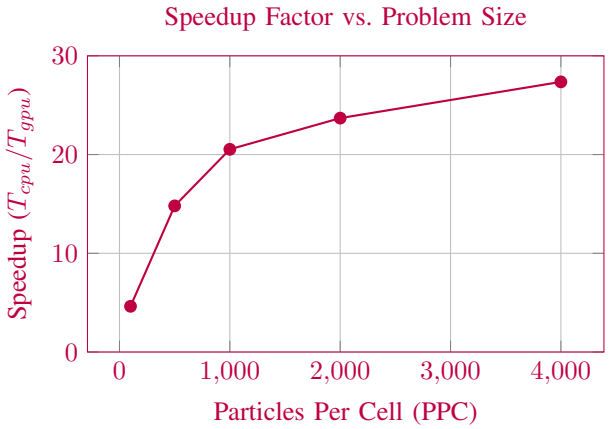


Fig. 6: Speedup factor scaling. The GPU efficiency improves with problem size, demonstrating that the architecture favors heavy workloads.

1) **Performance Gains:** The implementation achieves approximately 124 million particles per second at high load. Compared to typical single-core CPU implementations (around 5-10 Mpart/s), the gain is over an order of magnitude.
2) **Importance of Saturation:** The A100 GPU requires massive workloads to justify its use. Low PPC scenarios (e.g., 100) are inefficient. Scalability is excellent up to 4000 PPC.
3) **Correctness:** The use of atomics and warp reductions ensured data integrity without excessively sacrificing performance.

Future work could explore *Shared Memory* for current deposition using *grid tiling* techniques, which could reduce atomic contention in global memory for very high-density simulations. Additionally, implementing CUDA streams would allow overlapping diagnostic data copying with computation.

Acknowledgments

We acknowledge the Minho Advanced Computing Center (MACC) for providing access to the Deucalion supercomputer resources used in this work.

References

[1] C. K. Birdsall and A. B. Langdon, "Plasma Physics via Computer Simulation," CRC Press, 2004.
[2] NVIDIA Corporation, "CUDA C++ Programming Guide," v12.0, 2023.
[3] R. A. Fonseca et al., "OSIRIS: A Three-Dimensional, Fully Relativistic Particle in Cell Code for Modeling Plasma Based Accelerators," Springer, 2002.